Week 24 Graded Mini Project: Grounded Answers from a Small Knowledge Pack

Arpit Jain | Notebook and code: Week 24_Graded Mini Project_Jain.ipynb, rag.py (full outputs in the attached zip)

Task 1: Data prep (tiny corpus)

Topic: HR policies of one company, *Kestrel Data Labs* (a **fictional** company created for this project). Because the company is fictional, the LLM cannot know any of the answers from training, so every correct answer must come from retrieval. That makes groundedness and "don't know" behaviour easy to test. The 7 PDFs are generated by make_corpus.py, page by page, so each fact sits on a known page.

#	Document title	Source path	Pages	Words
1	Code of Conduct	corpus/05_code_of_conduct.pdf	2	243
2	IT and Information Security Policy	corpus/04_it_security_policy.pdf	2	247
3	Learning and Development Policy	corpus/07_learning_development_policy.pdf	2	180
4	Leave Policy	corpus/01_leave_policy.pdf	3	420
5	Performance and Promotion Policy	corpus/06_performance_promotion_policy.pdf	2	217
6	Remote and Hybrid Work Policy	corpus/02_remote_work_policy.pdf	2	259
7	Travel and Expense Policy	corpus/03_travel_expense_policy.pdf	2	254
	Total		15	1820

- **Extraction:** pypdf extracts text from each page, producing one LangChain Document per page. The running footer line is removed so it does not pollute the chunks.
- **Metadata on every chunk:** doc_title, source (path), page and chunk_id (for example 01_leave_policy-p2-c0). Pages are split one at a time, so a chunk never spans two pages and its [title#page] citation is always exact.

rag.py - loader with metadata

```
def load_corpus(corpus_dir=CORPUS_DIR):
    docs = []
    for pdf in sorted(Path(corpus_dir).glob("*.pdf")):
        reader = PdfReader(pdf)
        title = reader.metadata.title or pdf.stem
        for page_no, page in enumerate(reader.pages, start=1):
            text = page.extract_text() or ""
            # drop the running footer line ("<title> - Kestrel ... - Page n") so it doesn't pollute chunks
            lines = [l for l in text.splitlines() if "Kestrel Data Labs (fictional) - Page" not in l]
            text = re.sub(r"[ \t]+", " ", "\n".join(lines)).strip()
            docs.append(Document(page_content=text, metadata={
                "doc_title": title, "source": str(pdf.relative_to(HERE)), "page": page_no}))
    return docs
```

Task 2: Baseline pipeline

Pipeline: PyPDF loader → RecursiveCharacterTextSplitter (chunk_size 600, overlap 15% = 90 chars) → embeddings (OpenAI text-embedding-3-small or open-source BAAI/bge-small-en-v1.5) → FAISS → retriever (top-k = 4) → gemini-3.5-flash-lite (temperature 0 requested; this model uses fixed sampling and ignores it, so answers are cached for exact reproducibility) → citation-enforcing prompt → answer + citations[].

- **Runnable graph (LCEL):** condense → retrieve → prompt → LLM. The condense step uses the conversation memory to rewrite a follow-up such as "how many of *those* can I carry forward?" into a standalone search query. Without it, retrieval for follow-ups would search for the word "those".
- **Memory:** ConversationBufferMemory (langchain_classic.memory, its location since LangChain 1.0) stores each Q/A turn and passes the full history to both the condense step and the answer prompt. It is cleared after every 3-turn conversation.
- **Context labels:** each retrieved passage is labelled with the exact citation string the model must copy, for example [Leave Policy#2]. Citations are then parsed from the answer with a regex and returned as a list.
- **Output per question:** answer, citations=[...], used_chunks (IDs), used_pages, standalone_question, input and output tokens, and latency.

rag.py - the Runnable graph

```
def build_chain(vectorstore, prompt_id):
    llm = get_llm()
    retriever = vectorstore.as_retriever(search_kwargs={"k": TOP_K})

    def condense(x):
        if not x["chat_history"]:
```

```
            return {"text": x["question"], "usage": {}}
        msg = (CONDENSE | llm).invoke(x)
        return {"text": msg.text.strip(), "usage": msg.usage_metadata or {}}

    def generate(x):
        msg = (PROMPTS[prompt_id] | llm).invoke({"context": format_context(x["docs"]),
                                                 "chat_history": x["chat_history"], "question":
x["question"]})
        return msg

    return (
        RunnablePassthrough.assign(condensed=RunnableLambda(condense))
        | RunnablePassthrough.assign(docs=lambda x: retriever.invoke(x["condensed"]["text"]))
        | RunnablePassthrough.assign(llm_msg=RunnableLambda(generate))
    )
```

rag.py - one turn with ConversationBufferMemory

```
def ask(self, question):
    t0 = time.perf_counter()
    history = self.memory.load_memory_variables({})["chat_history"]
    out = self.chain.invoke({"question": question, "chat_history": history})
    latency = time.perf_counter() - t0
    answer = out["llm_msg"].text.strip()
    self.memory.save_context({"input": question}, {"output": answer})

    usage = out["llm_msg"].usage_metadata or {}
    cu = out["condensed"]["usage"]
    citations = sorted({f"{t.strip()}#{p}" for t, p in CITATION_RE.findall(answer)})
    return {
        "answer": answer,
        "citations": citations,
        "standalone_question": out["condensed"]["text"],
        "used_chunks": [d.metadata["chunk_id"] for d in out["docs"]],
        "used_pages": sorted({f"{d.metadata['doc_title']}#{d.metadata['page']}" for d in out["docs"]}),
        "input_tokens": usage.get("input_tokens", 0) + cu.get("input_tokens", 0),
        "output_tokens": usage.get("output_tokens", 0) + cu.get("output_tokens", 0),
        "answer_tokens": usage.get("output_tokens", 0) - (usage.get("output_token_details") or
{}).get("reasoning", 0),
        "reasoning_tokens": (usage.get("output_token_details") or {}).get("reasoning", 0),
        "latency_s": round(latency, 2),
    ...
```

Example output (R1, Q2 follow-up): standalone query = "How many days of annual leave can full-time employees carry forward to the next year, and what happens to the remaining days?"; citations = ['Leave Policy#2']; used_chunks = ['01_leave_policy-p2-c0', '01_leave_policy-p1-c1', '01_leave_policy-p2-c1', '01_leave_policy-p3-c0']; tokens = 778 in / 86 out; latency = 13.7s.

Task 3: Prompt templates (2 variants)

Both templates share the same grounding rules. They differ only in **output structure**:

Shared system rules (P1 and P2)

```
You are an HR policy assistant for Kestrel Data Labs. Use ONLY the context passages below; do not use
outside knowledge. Every factual sentence must end with a citation in the exact form [Document Title#page],
copied from the passage labels. If the context does not contain the answer, reply exactly: "I don't know
based on the provided policies." and add no citations.
```

	P1 - concise	P2 - reasoned (structured)
Extra instruction	Answer in at most 120 tokens (2-4 short sentences).	Format: 'Evidence:' with at most 2 bullets, each a short snippet with a citation, then 'Answer:' of at most 60 words. No other reasoning shown.
Idea	Shortest grounded answer	Point to the support first, then answer. This is visible structure, not private chain-of-thought
Avg output tokens (R1 vs R7)	42	92
Citation score (R1 vs R7)	0.95	0.85
Overall score (R1 vs R7)	0.99	0.91

R1 (P1) - Q7 "What are the eligibility criteria for promotion?"

```
To be eligible for promotion, an employee must have spent at least 18 months in their current level
[Performance and Promotion Policy#2]. Additionally, they must have received a rating of 4 or higher in each
of the last two review cycles [Performance and Promotion Policy#2].
```

R7 (P2) - Q7 "What are the eligibility criteria for promotion?"

```
Evidence:
- "To be eligible for promotion, an employee must have spent at least 18 months in their current level and
must have received a rating of 4 or higher in each of the last two review cycles." [Performance and
Promotion Policy#2]

Answer: To be eligible for promotion, employees must have spent at least 18 months in their current level
```

```
and received a rating of 4 or higher in each of the last two review cycles [Performance and Promotion Policy#2].
```

Prompt: P1 (concise). P1 averaged 42 output tokens with overall 0.99, and P2 averaged 92 tokens with overall 0.91 (citation 0.95 vs 0.85). P2's evidence bullets make answers easier to audit, but they did not improve accuracy enough to justify the longer output. P1 stays the default, and P2 is useful when an auditor needs to see the evidence.

Task 4: Experiments grid

Step 1 - retrieval benchmark (no LLM). Every chunking configuration was indexed with both embedding models, and the top-4 results for the 8 answerable questions were compared with the gold pages in questions.json.

chunk / overlap	chunks	OpenAI P@4	OpenAI R@4	OpenAI MRR	BGE P@4	BGE R@4	BGE MRR
300 / 10%	50	0.594	1.000	1.000	0.500	1.000	1.000
300 / 15%	50	0.594	1.000	1.000	0.500	1.000	1.000
300 / 20%	51	0.594	1.000	1.000	0.500	1.000	1.000
600 / 10%	29	0.312	1.000	1.000	0.281	1.000	1.000
600 / 15%	29	0.344	1.000	1.000	0.312	1.000	1.000
600 / 20%	29	0.344	1.000	1.000	0.312	1.000	1.000
1000 / 10%	15	0.250	1.000	0.917	0.250	1.000	1.000
1000 / 15%	15	0.250	1.000	0.917	0.250	1.000	1.000
1000 / 20%	15	0.250	1.000	0.917	0.250	1.000	1.000

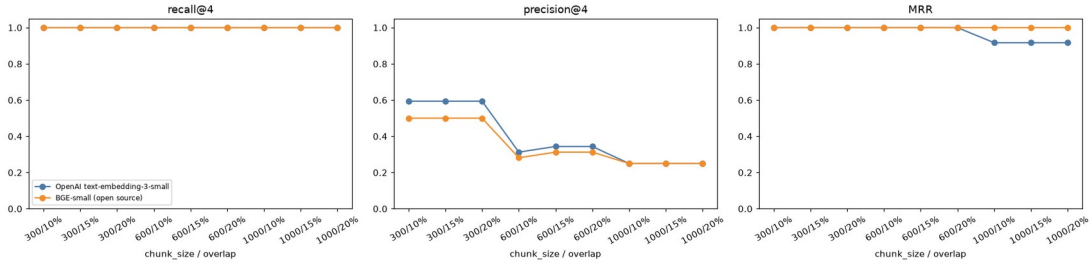


Figure 1 - Recall@4, Precision@4 and MRR by chunking configuration and embedding model

- **Chunking:** precision@4 ranges from 0.25 (1000 / 10%, openai) to 0.59 (300 / 10%, openai). With small chunks, the 4 retrieved passages are more likely to come from different pages, so precision drops. Large chunks (1000) are almost whole pages, so the top-4 contain fewer, broader passages. Overlap (10-20%) matters little in a corpus this small, because most facts fit inside a single sentence.
- **Embeddings:** averaged over all configurations, OpenAI reaches recall@4 1.00 / precision@4 0.39 / MRR 0.97, and BGE 1.00 / 0.35 / 1.00.

Embedding cost and latency (cold, uncached run on the baseline chunks):

Model	Dims	Embed chunks (s)	Per query (ms)	Tokens	Cost (this corpus)	Price / 1M tokens	Runs on
OpenAI text-embedding-3-small	1536	3.19	349.1	2709	$0.000054	0.02	OpenAI cloud API
BGE-small-en-v1.5 (local CPU)	384	0.62	16.2	2709	$0.000000	0 (self-hosted)	this laptop, no network

Step 2 - LLM runs, one factor at a time from the baseline (7 runs × 10 questions = 70 answers):

Run	chunk_size	overlap	embedding	prompt	factor changed
R1	600	15%	openai	P1	baseline
R2	300	15%	openai	P1	chunk 300
R3	1000	15%	openai	P1	chunk 1000
R4	600	10%	openai	P1	overlap 10%
R5	600	20%	openai	P1	overlap 20%
R6	600	15%	bge	P1	open-source embeddings
R7	600	15%	openai	P2	prompt P2

The chunk size (300 / 600 / 1000) and the overlap (10% / 15% / 20%) are each tested against the baseline. Embeddings and prompts are each A/B tested, with everything else held fixed.

Task 5: Question set

There are 10 questions: **6 factual, 2 follow-up / multi-hop, 2 boundary**. They are asked as three 3-turn conversations (A, B, C) plus one single turn (D). Memory is reset after each conversation.

ID	Conv/ turn	Type	Question	Gold page(s)	Expected answer
Q1	A1	factual	How many days of annual leave do full-time employees get each year?	Leave Policy#1	24 days per leave year, accruing at 2 days per completed month.
Q2	A2	follow-up	How many of those can I carry forward, and what happens to the rest?	Leave Policy#2	Up to 10 days carry forward; anything above 10 lapses on 31 December and is not paid out (except up to 15 days encashed on resignation).
Q3	A3	factual	What notice is needed to apply for planned leave?	Leave Policy#2	At least 7 calendar days in PeopleDesk; more than 5 consecutive working days needs 21 days' notice and manager + department head approval.
Q4	B1	factual	Which flight class can I book for a 4-hour domestic flight?	Travel and Expense Policy#1	Economy class (standard for flights up to 6 hours).
Q5	B2	factual	What is the daily allowance for international travel?	Travel and Expense Policy#2	USD 75 per day (not paid on days meals are provided).
Q6	B3	boundary	Does the company reimburse my gym membership?	-	Not covered by the policies -> should say it doesn't know.
Q7	C1	factual	What are the eligibility criteria for promotion?	Performance and Promotion Policy#2	At least 18 months in current level and a rating of 4 or higher in each of the last two review cycles.
Q8	C2	follow-up	Once I'm promoted to L5, which class can I fly on an 8-hour flight?	Travel and Expense Policy#1	Business class (flights over 6 hours, level L5 and above).
Q9	C3	boundary	What is the CEO's annual salary?	-	Not in the policies -> should say it doesn't know.
Q10	D1	factual	I clicked a link in a phishing email. How quickly must I report it, and to whom?	IT and Information Security Policy#2	Within 1 hour, to Security Operations at security@kestrel.example or hotline extension 4444.

- **Q2 (follow-up):** "those" refers to the annual-leave days from Q1, so it only works if memory is used.
- **Q8 (multi-hop):** asked after a promotion question. It combines the L5 level from the conversation with the travel policy.
- **Q6 and Q9 (boundary):** asked *inside* conversations, to check that the history does not tempt the model to over-reach.

Task 6: Logging

Every answer is appended to **logs/run_log.jsonl**, with a flat copy in logs/run_log.csv: one JSON object per answer, 70 rows in total. Each row contains:

- **Required:** run_id, question, answer, citations, used_chunks (chunk IDs) and used_pages (titles#page), input/output tokens and latency_s.
- **Configuration metadata:** chunk_size, overlap, embedding, prompt_id, llm, top_k, temperature, plus qid, conversation, turn, question type, the standalone (condensed) question and a timestamp.

run_id	qid	citations	used_chunks	tokens in/out	latency
R1	Q1	Leave Policy#1	01_leave_policy-p1-c1, 01_leave_policy-p2-c1, 01_leave_policy-p2-c0, 01_leave_policy-p3-c0	632/21	7.89s
R1	Q2	Leave Policy#2	01_leave_policy-p2-c0, 01_leave_policy-p1-c1, 01_leave_policy-p2-c1, 01_leave_policy-p3-c0	778/86	13.7s
R1	Q3	Leave Policy#2	01_leave_policy-p2-c0, 01_leave_policy-p3-c1, 01_leave_policy-p3-c0, 01_leave_policy-p1-c1	920/69	13.14s
R1	Q4	Travel and Expense Policy#1	03_travel_expense_policy-p1-c1, 03_travel_expense_policy-p1-c0, 03_travel_expense_policy-p2-c0, 02_remote_work_policy-p1-c1	636/53	6.81s
R1	Q5	Travel and Expense Policy#2	03_travel_expense_policy-p2-c0, 03_travel_expense_policy-p1-c0, 03_travel_expense_policy-p1-c1, 02_remote_work_policy-p2-c0	904/61	13.58s

One complete log record


```
{
 "run_id": "R1",
 "chunk_size": 600,
 "overlap": "15%",
 "embedding": "openai",
 "prompt_id": "P1",
 "llm": "gemini-3.5-flash-lite",
 "qid": "Q2",
 "question": "How many of those can I carry forward, and what happens to the rest?",
 "answer": "Employees may carry forward a maximum of 10 unused annual leave days [Leave Policy#2]. Any unused balance above 10 days lapses on 31 December and is not paid out, except upon resignation when up to 15 accrued days are encashed [Leave Policy#2].",
 "citations": [
```

```
  "Leave Policy#2"
 ],
 "used_chunks": [
  "01_leave_policy-p2-c0",
  "01_leave_policy-p1-c1",
  "01_leave_policy-p2-c1",
  "01_leave_policy-p3-c0"
 ],
 "input_tokens": 778,
 "output_tokens": 86,
 "latency_s": 13.7
}
```


Average latency per question ranged from 8.15s to 10.98s across runs. This includes the condense call for follow-up turns and the free-tier rate limiter (about 9 requests per minute). Total tokens per run are compared in Task 8.

Task 7: Manual evaluation

Criterion	Scale	Scoring rule
Relevance	0-2	2 = all key facts of the gold answer; 1 = some; 0 = wrong / off-topic
Groundedness	0-2	2 = every claim supported by retrieved chunks; 0 = unsupported content / cites unretrieved pages
Citation correctness	0-2	2 = citations present, all from retrieved chunks, incl. the gold page; 1 = partly; 0 = none / wrong
Conciseness	0-1	1 = within 120 tokens, no padding
Memory continuity	0-2	turns 2-3: 2 = uses (or is not confused by) prior turns correctly
Refusal correct?	0-1	boundary questions: 1 = says it doesn't know, no citations, nothing invented

Process: scoring.py first drafted every score by applying these rules to the gold answer key. It checks key facts, whether the cited pages were retrieved and match the gold page, the length, and refusals. Each answer was then read and the scores corrected by hand in logs/manual_scores.csv (70 of 70 rows marked reviewed).

Boundary handling ('Refusal correct?') per run:

Run	R1	R2	R3	R4	R5	R6	R7
Q6 (Does the company reimburse my gym memb)	✓	✓	✓	✓	✓	✓	✓
Q9 (What is the CEO's annual salary?)	✓	✓	✓	✓	✓	✓	✓

Memory continuity on the follow-up questions (0-2):

Run	R1	R2	R3	R4	R5	R6	R7
Q2	2	2	2	2	2	2	2
Q8	2	2	2	2	2	2	2

Task 8: Summary report

Runs table: configuration → average scores (each criterion normalised to 0-1; overall = mean of the 6):

Run	Config (chunk/ovl/emb/prompt)	Rel	Grnd	Cite	Conc	Mem	Refus	Overall	Points	Tok/run
R1	600/15%/OpenAI/P1	1.00	1.00	0.95	1.00	1.00	1.00	0.99	83 / 84	8310
R2	300/15%/OpenAI/P1	1.00	1.00	0.85	1.00	1.00	1.00	0.97	81 / 84	6404
R3	**1000/15%/OpenAI/P1**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**84 / 84**	**10873**
R4	600/10%/OpenAI/P1	1.00	1.00	1.00	1.00	1.00	1.00	1.00	84 / 84	8338
R5	600/20%/OpenAI/P1	1.00	1.00	1.00	1.00	1.00	1.00	1.00	84 / 84	8261
R6	600/15%/BGE/P1	1.00	1.00	1.00	1.00	1.00	1.00	1.00	84 / 84	8006
R7	600/15%/OpenAI/P2	1.00	1.00	0.85	0.60	1.00	1.00	0.91	77 / 84	10271

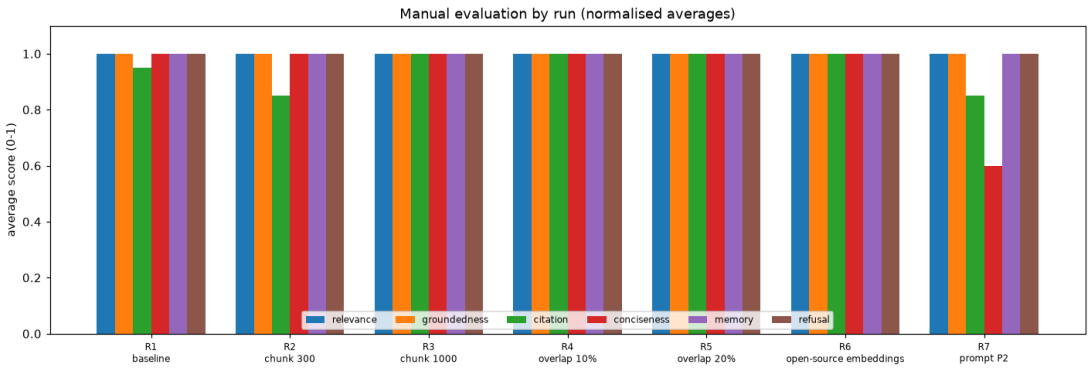


Figure 2 - Average scores per run

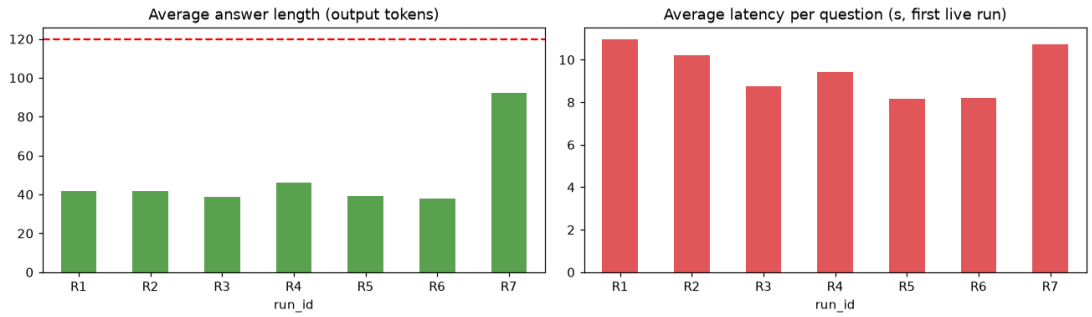


Figure 3 - Answer length and latency per run

Example Q/A pairs:

Win: follow-up resolved through memory - R3, Q8: "Once I'm promoted to L5, which class can I fly on an 8-hour flight?"

```
Standalone query: What are the flight class entitlements for an L5 employee on an 8-hour flight?
Retrieved: 01_leave_policy-p2-c0, 03_travel_expense_policy-p1-c0, 07_learning_development_policy-p2-c0,
06_performance_promotion_policy-p2-c0
Answer: For flights longer than 6 hours, employees at level L5 and above may book business class [Travel
and Expense Policy#1].
```

Scores: relevance=2, groundedness=2, citation=2, conciseness=1, memory=2. Note: standalone query: What are the flight class entitlements for an L5 employee on an 8-hour flight?

Win: correct refusal on an out-of-corpus question - R3, Q6: "Does the company reimburse my gym membership?"

```
Standalone query: Does the company reimburse my gym membership?
Retrieved: 07_learning_development_policy-p2-c0, 03_travel_expense_policy-p2-c0, 02_remote_work_policy-p2-
c0, 03_travel_expense_policy-p1-c0
Answer: I don't know based on the provided policies.
```

Scores: relevance=2, groundedness=2, citation=2, conciseness=1, memory=2, refusal_correct=1. Note: correct refusal

Failure: weakest answer across all runs - R7, Q2: "How many of those can I carry forward, and what happens to the rest?"

```
Standalone query: How many days of annual leave can full-time employees carry forward each year, and what
happens to the remaining days?
Retrieved: 01_leave_policy-p2-c0, 01_leave_policy-p1-c1, 01_leave_policy-p2-c1, 01_leave_policy-p3-c0
Answer: Evidence:
- Employees may carry forward a maximum of 10 unused annual leave days into the next leave year. [Leave
Policy#2#2]
- Any unused balance above 10 days lapses on 31 December and is not paid out, except on resignation, when
up to 15 accrued days are encashed at the basic salary rate. [Leave Policy#2#2]
Answer: You may carry forward a maximum of 10 unused annual leave days [Leave Policy#2#2]. Any balance
above 10 days lapses on 31 December and is not paid out, except upon resignation where up to 15 accrued
days are encashed at the basic salary rate [Leave Policy#2#2].
```

Scores: relevance=2, groundedness=2, citation=1, conciseness=0, memory=2. Note: citation format includes the chunk number ([Title#page#chunk]) - page is correct; standalone query: How many days of annual leave can full-time employees carry forward each year, and what happens to the remaining days?

Recommendation: chunking 600 / 20% + Open-source BGE-small + P1 (concise)

- **Chunking: 600 characters, 20% overlap** (run R5). Among the chunking runs it scored the highest overall (1.00), with retrieval MRR 1.00. Smaller chunks give more precise passages, but a policy rule and its condition can be split across chunks. Larger chunks keep rules together, but they put more unrelated text into the prompt (total tokens per run: R1=8310, R2=6404, R3=10873, R4=8338, R5=8261).
- **Embeddings: Open-source BGE-small.** In the end-to-end runs, OpenAI scored 0.99 and BGE 1.00. Across all 9 chunking configurations, average retrieval was: OpenAI recall@4 1.00 / MRR 0.97, against BGE 1.00 / 1.00. OpenAI costs $0.02 per 1M tokens ($0.000054 for this corpus) and takes about 349 ms per query over the network. BGE is free, runs locally in about 16 ms per query, and keeps policy text inside the company. Since quality is essentially equal, the free and private model wins.
- **Prompt: P1 (concise).** P1 averaged 42 output tokens with overall 0.99, and P2 averaged 92 tokens with overall 0.91 (citation 0.95 vs 0.85). P2's evidence bullets make answers easier to audit, but they did not improve accuracy enough to justify the longer output. P1 stays the default, and P2 is useful when an auditor needs to see the evidence.

Reproducibility:

- Versions and configuration are printed in notebook section 0 (LLM gemini-3.5-flash-lite, temperature 0 requested but ignored by this model, top_k 4, seed 42, exact FAISS search; hidden reasoning tokens are logged separately). requirements.txt pins every package.
- The corpus is regenerated deterministically by make_corpus.py, and the questions and gold key are in questions.json.

- LLM responses are cached in cache/llm_cache.sqlite and embeddings in cache/embeddings/, so re-running the notebook reproduces every answer without new API calls.
- Run: install requirements.txt, copy .env.example to .env and add the keys, then execute the notebook (see the README). The submitted logs and cache reproduce every result.